Week 2 - Sorting Algos

15 October 2026 18:58

Selection sort

$[3_a, 2, 7, 1, 8, 3_b, 5, 10]$ — First check minimum element (len(list))

3 (Index)

$[1, 2, 7, 3_a, 8, 3_b, 5, 10]$ — Swap min element with first elem.

n = length of array

n swap

$n-1$ swap

$n-2$

$O(n^2)$ — worst, Avg, Best

Insertion Sort

$[3_a, 2, 7, 1, 8, 3_b, 5, 10]$

$[1, 2, 3, 4, 5]$

$2, 3_a, 7, 1, 8, 3_b, 5, 10$

$2, 3_a, 1, 7, 8, \ldots$

$2, 1, 3_a, 7, 8, \ldots$

$1, 2, 3_a, 7, 8, 3_b, 5, 10$

$1, 2, 3_a, 3_b, 7, 8, 5, 10$

$1, 2, 3_a, 3_b, 5, 7, 8, 10$

$O(n^2)$ — worst, Avg

$O(n)$ — Best case (given a sorted list)

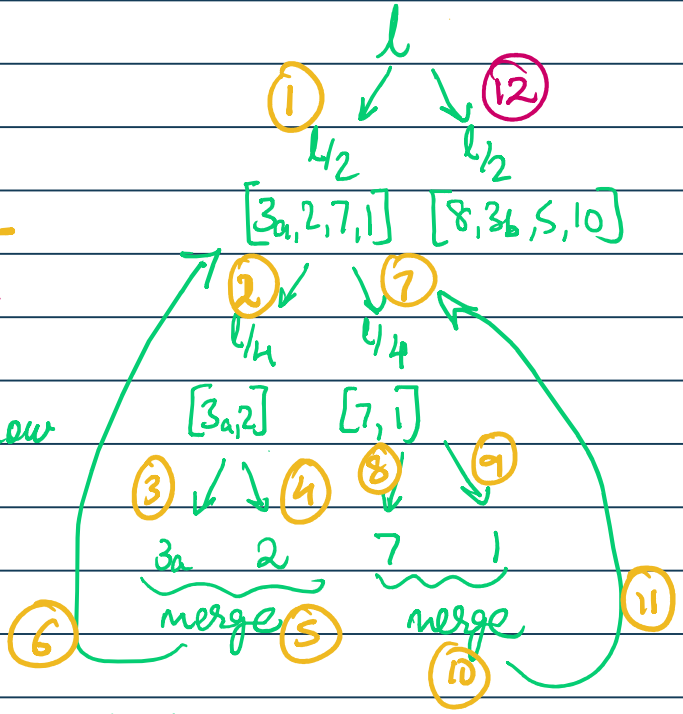
In-place, Stable

S.C $\Rightarrow O(1)$

Merge sort

$[3_a, 2, 7, 1, 8, 3_b, 5, 10]$

```
mid = len(l) // 2
left = l[: mid]
right = l[mid: ]
# merge left, right
# while merging you know
  left, right are sorted.
return
```



either implement
merge inside merge_sort()
or create a auxiliary function
outside merge_sort() called merge(left, right)

$[3_a, 2, 7, \overset{3}{1}, 8, 3_b, 5, 10]$